

# SKILL-8

## SESSION-15

Task-1: TO Parse Combined Redirection, Duplicate File Descriptors, Merge Streams, Verify Ordering, Handle Edge Cases, Test Integrated Outputs

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>

#define MAX_INPUT 256

int main() {
    char input[MAX_INPUT];
    char filename[MAX_INPUT];

    printf("Combined Redirection Demonstration\n");
    printf("-----\n");

    printf("Enter command with combined redirection: ");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading input.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';

    /*
     * Parse combined redirection.
     * Expected format:
     * command > filename 2>&1
     */
    char *redirect = strstr(input, ">");

    if (redirect == NULL) {
        printf("Error: Output redirection '>' not found.\n");
        return 1;
    }

    char *merge = strstr(input, "2>&1");

    if (merge == NULL) {
        printf("Error: Combined stderr redirection '2>&1' not found.\n");
        return 1;
    }
}
```

```

/* Extract filename between > and 2>&1 */
int filename_length = merge - redirect - 1;

if (filename_length <= 0) {
    printf("Error: No output filename specified.\n");
    return 1;
}

strncpy(filename, redirect + 1, filename_length);
filename[filename_length] = '\0';

/* Remove leading spaces */
char *start = filename;
while (*start == ' ') {
    start++;
}

if (start != filename) {
    memmove(filename, start, strlen(start) + 1);
}

/* Remove trailing spaces */
int len = strlen(filename);
while (len > 0 && filename[len - 1] == ' ') {
    filename[len - 1] = '\0';
    len--;
}

if (strlen(filename) == 0) {
    printf("Error: Invalid output filename.\n");
    return 1;
}

printf("Parsed output file: %s\n", filename);
printf("Combined redirection detected: > file 2>&1\n");

/*
 * Open/create output file.
 */
int fd = open(filename, O_WRONLY | O_CREAT | O_TRUNC, 0644);

```

```

    return 1;
}

/*
 * Save original stdout and stderr.
 */
int saved_stdout = dup(STDOUT_FILENO);
int saved_stderr = dup(STDERR_FILENO);

if (saved_stdout == -1 || saved_stderr == -1) {
    perror("dup");
    close(fd);

    if (saved_stdout != -1)
        close(saved_stdout);

    if (saved_stderr != -1)
        close(saved_stderr);

    return 1;
}

/*
 * Redirect stdout to the file.
 */
if (dup2(fd, STDOUT_FILENO) == -1) {
    perror("stdout redirection failed");
    close(fd);
    close(saved_stdout);
    close(saved_stderr);
    return 1;
}

/*
 * Duplicate stdout's file descriptor onto stderr.
 * Now stdout and stderr both point to the same file.
 */
if (dup2(STDOUT_FILENO, STDERR_FILENO) == -1) {
    perror("stderr redirection failed");
    close(fd);
    close(saved_stdout);
    close(saved_stderr);
}

```

```

/*
if (dup2(saved_stdout, STDOUT_FILENO) == -1) {
    perror("Restoring stdout failed");
}

if (dup2(saved_stderr, STDERR_FILENO) == -1) {
    perror("Restoring stderr failed");
}

close(saved_stdout);
close(saved_stderr);

printf("stdout and stderr restored successfully.\n");
printf("Combined redirection completed successfully.\n");

/*
 * Verify the integrated output.
 */
FILE *verify = fopen(filename, "r");
if (verify == NULL) {
    perror("Verification failed");
    return 1;
}

printf("\nVerified integrated output from %s:\n", filename);
printf("-----\n");

char buffer[128];
while (fgets(buffer, sizeof(buffer), verify) != NULL) {
    printf("%s", buffer);
}

fclose(verify);

printf("-----\n");
printf("Both streams were successfully merged.\n");
return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ nano skill8s15task1.c
ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill8s15task1.c -o skill8s15task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill8s15task1

```

Combined Redirection Demonstration

```

-----
Enter command with combined redirection: echo Hello > combined.txt 2>&1
Parsed output file: combined.txt
Combined redirection detected: > file 2>&1
stdout and stderr restored successfully.
Combined redirection completed successfully.

```

Verified integrated output from combined.txt:

- ```

-----
1. Standard output message.
2. Standard error message.
3. Another standard output message.
4. Another standard error message.
-----

```

Both streams were successfully merged.

```

ruhi0404@Ruhitha:~/Operating_Systems$ cat combined.txt

```

- ```

1. Standard output message.
2. Standard error message.
3. Another standard output message.
4. Another standard error message.

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ ./skill8s15task1

```

Combined Redirection Demonstration

```

-----
Enter command with combined redirection: ls > result.txt 2>&1
Parsed output file: result.txt
Combined redirection detected: > file 2>&1
stdout and stderr restored successfully.
Combined redirection completed successfully.

```

Verified integrated output from result.txt:

- ```

-----
1. Standard output message.
2. Standard error message.
3. Another standard output message.
4. Another standard error message.
-----

```

Both streams were successfully merged.

```

ruhi0404@Ruhitha:~/Operating_Systems$ |

```

### Observation:

The program successfully parses combined output redirection using > and 2>&1.

Standard output is redirected to the specified file using dup2( ), and the standard error descriptor is duplicated onto the standard output descriptor so that both streams use the same file. Messages from stdout and stderr are written alternately and verified in the expected order. The program also handles missing redirection operators and filenames and restores both stdout and stderr after the operation.

### Task-2: To Parse Complex Commands, Combine Pipes and Redirections, Resolve Execution Order, Validate Syntax, Generate Execution Plans, Test Scenarios

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/wait.h>

#define MAX_INPUT 512
#define MAX_PARTS 10

void trim(char *str) {
    int start = 0;
    int end = strlen(str) - 1;

    while (str[start] == ' ' || str[start] == '\t')
        start++;

    while (end >= start && (str[end] == ' ' || str[end] == '\t')) {
        str[end] = '\0';
        end--;
    }

    if (start > 0)
        memmove(str, str + start, strlen(str + start) + 1);
}

int main() {
    char input[MAX_INPUT];
    char command_copy[MAX_INPUT];

    printf("Complex Command Parser and Execution Planner\n");
    printf("-----\n");

    printf("Enter complex command:\n");

    if (fgets(input, sizeof(input), stdin) == NULL) {
        printf("Error reading command.\n");
        return 1;
    }

    input[strcspn(input, "\n")] = '\0';
}
```

^G Help      ^O Write Out      ^W Where Is      ^K Cut      ^T Execute

```

/* -----
 * STEP 2: Split command into pipeline stages
 * -----
 */

char *stages[MAX_PARTS];
int stage_count = 0;

char *token = strtok(command_copy, "|");
while (token != NULL && stage_count < MAX_PARTS) {
    trim(token);
    if (strlen(token) == 0) {
        printf("Syntax Error: Empty pipeline stage.\n");
        return 1;
    }
    stages[stage_count++] = token;
    token = strtok(NULL, "|");
}

if (stage_count == 0) {
    printf("Error: No command found.\n");
    return 1;
}

/* -----
 * STEP 3: Display execution plan
 * -----
 */

printf("\nExecution Plan\n");
printf("-----\n");

printf("Number of pipeline stages: %d\n", stage_count);

for (int i = 0; i < stage_count; i++) {
    printf("Stage %d: %s\n", i + 1, stages[i]);
}

```

^G Help                      ^O Write Out                      ^W Where Is                      ^K Cut                      ^T Execute  
 ^X Exit                      ^R Read File                      ^\ Replace                      ^U Paste                      ^J Justify

```

char input_file[128] = "";
char output_file[128] = "";
char error_file[128] = "";

for (int i = 0; i < stage_count; i++) {
    /* Input redirection */
    char *in = strstr(stages[i], "<");
    if (in != NULL) {
        char temp[128];
        if (sscanf(in + 1, "%127s", temp) != 1) {
            printf("Syntax Error: Missing input file.\n");
            return 1;
        }
        strcpy(input_file, temp);
        *in = '\0';
        trim(stages[i]);
    }

    /* stderr redirection */
    char *err = strstr(stages[i], "2>");
    if (err != NULL) {
        char temp[128];
        if (sscanf(err + 2, "%127s", temp) != 1) {
            printf("Syntax Error: Missing error file.\n");
            return 1;
        }
        strcpy(error_file, temp);
        *err = '\0';
        trim(stages[i]);
    }

    /* stdout redirection */
    char *out = strstr(stages[i], ">");
}

```

```

/* -----
 * STEP 8: Parent closes pipes
 * -----
 */

for (int i = 0; i < stage_count - 1; i++) {
    close(pipes[i][0]);
    close(pipes[i][1]);
}

/* -----
 * STEP 9: Wait for all processes
 * -----
 */

int success = 1;
for (int i = 0; i < stage_count; i++) {
    int status;
    waitpid(pids[i], &status, 0);

    if (!WIFEXITED(status) ||
        WEXITSTATUS(status) != 0) {
        success = 0;
    }
}

printf("\n-----\n");

if (success)
    printf("Complex command executed successfully.\n");
else
    printf("Complex command completed with errors.\n");

printf("Execution plan completed.\n");

return 0;

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ ./skill8s15task2
Complex Command Parser and Execution Planner
-----
Enter complex command:
nano input.txt

Execution Plan
-----
Number of pipeline stages: 1
Stage 1: nano input.txt

Execution Order:
1. Execute: nano input.txt

Redirection Plan
-----

-----
Complex command executed successfully.
Execution plan completed.
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill8s15task2
Complex Command Parser and Execution Planner
-----
Enter complex command:
cat < input.txt | grep Operating > result.txt 2> errors.txt

Execution Plan
-----
Number of pipeline stages: 2
Stage 1: cat < input.txt
Stage 2: grep Operating > result.txt 2> errors.txt

Execution Order:
1. Execute: cat < input.txt
2. Execute: grep Operating > result.txt 2> errors.txt

Redirection Plan
-----
stdin  <- input.txt
stdout -> result.txt
stderr -> errors.txt

-----
Complex command executed successfully.
Execution plan completed.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

**Observation:**

The program successfully parses complex commands containing pipes and input, output, and error redirections. Pipeline stages are identified and an execution plan is

generated before execution. Pipes are created to transfer data between processes, while `dup2()` connects the appropriate standard streams to files or pipes. The program validates invalid pipeline syntax and missing filenames, launches the required processes using `fork()` and `execvp()`, and coordinates their completion using `waitpid()`. Valid and error scenarios were tested successfully.

## SESSION-16:

**Task-1:** To Detect Background Jobs, Launch Non-Blocking Processes, Return Prompt Immediately, Store Job Information, Monitor Execution, Test Background Tasks

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>
#include <signal.h>

#define MAX_JOBS 10
#define MAX_INPUT 256
#define MAX_ARGS 20

struct Job {
    int job_id;
    pid_t pid;
    char command[MAX_INPUT];
    int running;
};

struct Job jobs[MAX_JOBS];
int job_count = 0;

/* Check whether the command ends with '&' */
int is_background(char *command) {
    int len = strlen(command);

    while (len > 0 &&
           (command[len - 1] == ' ' ||
            command[len - 1] == '\t')) {
        len--;
    }

    if (len > 0 && command[len - 1] == '&') {
        command[len - 1] = '\0';
        return 1;
    }

    return 0;
}

/* Display stored jobs */
void display_jobs() {
```

```

GNU nano 7.2
}
/* Launch a background process */
void launch_background(char *command) {
    if (job_count >= MAX_JOBS) {
        printf("Error: Maximum number of jobs reached.\n");
        return;
    }

    char command_copy[MAX_INPUT];
    strcpy(command_copy, command);

    pid_t pid = fork();

    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        /* Convert command into arguments */
        char *args[MAX_ARGS];
        int argc = 0;

        char *token = strtok(command_copy, " ");

        while (token != NULL &&
               argc < MAX_ARGS - 1) {
            args[argc++] = token;
            token = strtok(NULL, " ");
        }

        args[argc] = NULL;

        if (argc == 0)
            exit(1);

        /* Execute command */
        execvp(args[0], args);
    }
}

```

^G Help      ^O Write Out      ^W Where Is      ^K Cut

```

    args[argc] = NULL;

    if (argc == 0)
        exit(1);

    execvp(args[0], args);

    perror("execvp");
    exit(1);
}

/* Foreground process is waited for */
waitpid(pid, NULL, 0);

printf("Foreground command completed.\n");
}

int main() {
    char input[MAX_INPUT];

    printf("Background Job Management Demonstration\n");
    printf("-----\n");

    printf("Commands:\n");
    printf("  command &    -> Run in background\n");
    printf("  jobs          -> Display background jobs\n");
    printf("  monitor       -> Check job status\n");
    printf("  exit          -> Exit program\n");

    while (1) {
        /* Check whether background jobs have completed */
        monitor_jobs();

        printf("\nmyshe11> ");
        fflush(stdout);

        if (fgets(input, sizeof(input), stdin) == NULL)
            break;

        input[strcspn(input, "\n")] = '\0';
    }
}

```

^G Help      ^O Write Out      ^W Where Is      ^K Cut      ^T Execute      ^C Local
 ^X Exit      ^R Read File      ^\_ Replace      ^U Paste      ^J Justify      ^\_ Go To



```

        display_jobs();
        continue;
    }

    /* Monitor jobs */
    if (strcmp(input, "monitor") == 0) {
        monitor_jobs();
        continue;
    }

    /* Detect background command */
    if (is_background(input)) {

        if (strlen(input) == 0) {
            printf("Error: No command before '&'.\n");
            continue;
        }

        launch_background(input);

        /*
         * Parent immediately returns to prompt.
         */
        continue;
    }

    /* Otherwise run as foreground command */
    launch_foreground(input);
}

/*
 * Final monitoring before exit.
 */
printf("\nChecking background jobs before exit...\n");

monitor_jobs();

printf("Exiting background job manager.\n");

return 0;
}

```

```

ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill8s16task1.c -o skill8s16task1
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill8s16task1
Background Job Management Demonstration
-----
Commands:
  command &    -> Run in background
  jobs         -> Display background jobs
  monitor      -> Check job status
  exit         -> Exit program

myshell> sleep 5 &
[1] 1278 started in background: sleep 5

myshell> jobs

Background Jobs
-----
[1] PID=1278  sleep 5    Running

[1] PID=1278 completed: sleep 5

myshell> jobs

Background Jobs
-----
[1] PID=1278  sleep 5    Completed

myshell> sleep 3 &
[2] 1281 started in background: sleep 3

myshell> jobs

Background Jobs
-----
[1] PID=1278  sleep 5    Completed
[2] PID=1281  sleep 3    Running

[2] PID=1281 completed: sleep 3

myshell> &
Error: No command before '&'.

myshell> exit

Checking background jobs before exit...
Exiting background job manager.

```

### Observation:

The program successfully detects commands ending with the background operator & and launches them using `fork()` without blocking the parent process. The parent immediately returns to the shell prompt while storing the job ID, process ID, command, and execution status in a job table. Background processes are monitored using `waitpid()` with the `WNOHANG` option, allowing their completion status to be checked without blocking the shell. Multiple background jobs and foreground commands were tested successfully.

Task-2: To Create Job Table, Store Process Groups, Track States, Update Status, Remove Completed Jobs, Validate Job Records

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

#define MAX_JOBS 10
#define MAX_COMMAND 256

/* Job states */
enum JobState {
    RUNNING,
    COMPLETED
};

/* Job record */
struct Job {
    int job_id;
    pid_t pid;
    pid_t pgid;
    char command[MAX_COMMAND];
    enum JobState state;
};

struct Job job_table[MAX_JOBS];
int job_count = 0;

/* Convert state to readable text */
const char *get_state_name(enum JobState state) {
    if (state == RUNNING)
        return "Running";

    return "Completed";
}

/* Add a new job to the job table */
void add_job(pid_t pid, pid_t pgid, char *command) {
    if (job_count >= MAX_JOBS) {
        printf("Job table is full.\n");
        return;
    }
}
```

```

    for (int i = 0; i < job_count; i++) {

        printf("%d\t%d\t%d\t%-10s\t%s\n",
               job_table[i].job_id,
               job_table[i].pid,
               job_table[i].pgid,
               get_state_name(job_table[i].state),
               job_table[i].command);

    }

    printf("-----\n");
}

/* Update the state of every job */
void update_job_status() {

    for (int i = 0; i < job_count; i++) {

        if (job_table[i].state == COMPLETED)
            continue;

        int status;

        pid_t result = waitpid(
            job_table[i].pid,
            &status,
            WNOHANG
        );

        if (result == job_table[i].pid) {

            job_table[i].state = COMPLETED;

            printf("\nJob [%d] completed: %s\n",
                   job_table[i].job_id,
                   job_table[i].command);

        }

    }

}

/* Remove completed jobs from the job table */
void remove_completed_jobs() {

```

```

        printf("cannot create job: job table is full.\n");
        return;
    }

    pid_t pid = fork();
    if (pid < 0) {
        perror("fork");
        return;
    }

    if (pid == 0) {
        /*
         * Create a new process group.
         * The child becomes the leader of its group.
         */
        if (setpgid(0, 0) == -1) {
            perror("setpgid");
            exit(1);
        }

        char command_copy[MAX_COMMAND];
        strcpy(command_copy, command);

        char *args[20];
        int argc = 0;

        char *token = strtok(command_copy, " ");

        while (token != NULL && argc < 19) {

            args[argc] = token;
            argc++;

            token = strtok(NULL, " ");
        }

        args[argc] = NULL;

        if (argc == 0)
            exit(1);
    }

```

```

        if (len > 0 && command[len - 1] == '&') {
            command[len - 1] = '\0';
            /* Remove trailing spaces */
            len = strlen(command);
            while (len > 0 &&
                   (command[len - 1] == ' ' ||
                    command[len - 1] == '\t')) {
                command[len - 1] = '\0';
                len--;
            }

            if (strlen(command) == 0) {
                printf("Error: No command before '&'.\n");
                continue;
            }

            launch_job(command);
            continue;
        }

        printf("Error: Use '&' to start a background job.\n");
    }

    /*
     * Update states before exiting.
     */
    update_job_status();

    printf("\nFinal Job Table:\n");
    display_jobs();

    printf("Job manager terminated.\n");

    return 0;
}

```

|      |           |          |       |     |
|------|-----------|----------|-------|-----|
| Help | Write Out | Where Is | Cut   | Exe |
| Exit | Read File | Replace  | Paste | Jus |

```

ruhi0404@Ruhitha:~/Operating_Systems$ gcc skill8s16task2.c -o skill8s16task2
ruhi0404@Ruhitha:~/Operating_Systems$ ./skill8s16task2
Job Table and Process Group Management
-----
Commands:
  command &  -> Start background job
  jobs       -> Display job table
  update     -> Update job states
  validate   -> Validate job records
  clean      -> Remove completed jobs
  exit       -> Exit

job-shell> sleep 5 &

Job [1] started successfully.
PID   : 1353
PGID  : 1353
State: Running

job-shell> jobs

Job Table
-----
ID      PID      PGID      State      Command
-----
1       1353     1353     Running    sleep 5
-----

Job [1] completed: sleep 5

job-shell> validate

Validating Job Records
-----
All job records are valid.

job-shell> sleep 3 &

Job [2] started successfully.
PID   : 1363
PGID  : 1363
State: Running

job-shell> exit

Job [2] completed: sleep 3

```

```

job-shell> jobs

Job Table
-----
ID      PID      PGID      State      Command
-----
1       1353     1353     Running    sleep 5
-----

Job [1] completed: sleep 5

job-shell> validate

Validating Job Records
-----
All job records are valid.

job-shell> sleep 3 &

Job [2] started successfully.
PID   : 1363
PGID  : 1363
State: Running

job-shell> exit

Job [2] completed: sleep 3

Final Job Table:

Job Table
-----
ID      PID      PGID      State      Command
-----
1       1353     1353     Completed  sleep 5
2       1363     1363     Completed  sleep 3
-----

Job manager terminated.
ruhi0404@Ruhitha:~/Operating_Systems$ |

```

#### Observation:

The program successfully creates and maintains a job table containing the job ID, process ID, process group ID, command, and current state of each background process. Process groups are created using `setpgid()`. The state of each job is monitored using `non-blocking waitpid()` with `WNOHANG`, allowing completed processes to be detected without blocking the shell. Completed jobs can be removed from the job table, while the validation function checks the stored PID, PGID, and command information. Multiple background jobs were tested successfully.